

Expert Extraction Technical Report

Methods

Hardware and software requirements

This project used an NVIDIA RTX A4000 rented from vast.ai¹. The A4000 has 16 GB VRAM, which is only enough for inference of a ~7B model at 16-bit precision. GPUs with larger memory, such as the A100, can allow experimenting with larger models, but are only available at several times the price. A PyTorch container from vast.ai, including Python 3.12 and CUDA 12.4, was used. About 20 GB disk was used to cache the downloaded model and dataset.

Model

The model used was an instruction fine-tuned OLMoE², available on Hugging Face as allenai/OLMoE-1B-7B-0924-Instruct. OLMoE is an open-weight, open-source mixture-of-experts (MoE) model, with 7B total parameters and 1B active parameters. It consists of 16 layers of attention and MLP. Each MLP block has 64 experts, 8 of which are activated per token. Experts use the SwiGLU³ activation function. Tokens have dimension 2048, while experts have intermediate dimension 1024.

Because MoE is mainly for scaling AI models while maintaining inference speed, most MoEs are large and will cause an out-of-memory (OOM) error when loaded. Therefore, more popular models such as gpt-oss-20b (21B parameters) or Qwen1.5-MoE-A2.7B (14B parameters) were not used.

MoE quantization

OLMoE used 14 GB out of 16 GB VRAM on load. The remaining memory is insufficient for distillation. The bitsandbytes⁴ library offers 4-bit quantization with QLoRA, but it reduces consumption by <1 GB when directly used. This is because OLMoE's experts use `torch.nn.functional.linear` to compute the forward pass, but bitsandbytes works by replacing `torch.nn.Linear` with its quantized version `Linear4bit`. To

1 <https://cloud.vast.ai>

2 Muennighoff et al. (2025). OLMoE: Open Mixture-of-Experts Language Models. <https://arxiv.org/abs/2409.02060>

3 Shazeer (2020). GLU Variants Improve Transformer. <https://arxiv.org/abs/2002.05202>

4 <https://huggingface.co/docs/transformers/v5.17.0/quantization/bitsandbytes>

address this, OLMoE’s experts were replaced with a custom version using the Linear module. This reduces consumption to 7 GB on load.

Expert extraction

Two methods were attempted. The first method was simply to extract the first expert from each layer. The second was to average the weights of all experts. Both produced poor results (see [Results](#)).

To produce the student model, each layer’s experts were replaced with an MLP whose configuration is identical to an expert (2048 input dimension, 1024 intermediate dimension), then the expert weights are copied to the MLP.

The student model has 575M parameters, less than 10% of the teacher’s.

LoRA distillation

Low-rank adaptation configuration mostly followed standard practice⁵: rank 16, alpha 32, dropout 0, batch size 2, gradient accumulation steps 8. However, whereas standard practice targets both attention and MLP parameters, I was only able to target attention parameters due to a bug in the code. At batch size 2, 11.4 GB memory was used during distillation; batch size 4 caused OOM. With only attention targeted, there are 4M trainable parameters, already 0.7% of total parameters. Because targeting MLP later will further increase this, rank 16 was preferred over rank 32.

The dataset used was the Colossal Clean Crawled Corpus (C4)⁶.

Loss was defined as the KL divergence of the student distribution from the teacher distribution. A temperature of 2 was tentatively used for both distributions⁷.

Results and discussion

A full distillation run was not performed due to the poor initial loss of the student model. When temperature was set to 1, the initial loss was around 10. Because OLMoE’s vocabulary size is about 50,000 and $\ln(50000)=10.8$, the student was no better than a uniform distribution. Such a model will likely take too long to distill

5 <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide>

6 Raffel et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. <https://arxiv.org/abs/1910.10683>

7 Hinton et al. (2015). Distilling the Knowledge in a Neural Network. <https://arxiv.org/abs/1503.02531>

because it effectively needs to model language using only LoRA's parameters, which are a small fraction of the full model's parameters.

The next step is to investigate which experts receive tokens more often. The model can be set to output router logits, which decide which 8 experts are selected and how much they are weighed. From there, the most frequently used expert can be extracted, or a weighted sum of experts can be performed, weighed by the mean router weight.

AI usage disclosure

AI models were used for advice and step-by-step instructions. They were not allowed to generate code blocks because these can be copied and pasted, but they could use lines of code in their explanations. No coding agents were used.